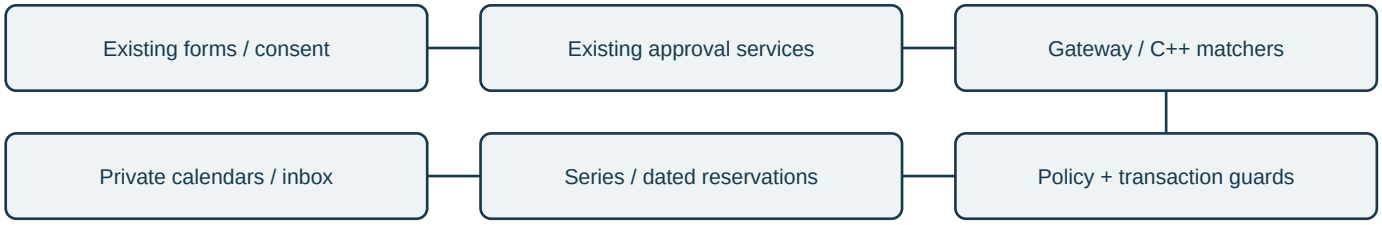
ITERATION 49 / UNIVERSITY RESOURCE ALLOCATION

Recurring Reservations and Approval Policies

Objective: extend UC-STU-02/03 using the existing Flask/PostgreSQL workflows and C++ matchers, with explicit calendar dates, student-selected approval consent and accountable resource priority.

Implemented behavior

Students preview bounded weekly dates, select one configured semester and choose all occurrences required or partial approval allowed. Rooms prefer a consistent eligible selection. Staff decide exact dates; every occurrence retains its outcome and decision details. Equipment reservations retain full quantities, actual physical handovers, partial verified returns, extensions and overdue obligations. Individual/future-series cancellation never erases past use.



Responsibilities and database

Component	Responsibility
recurrence / reservation_policy	Reuse date/timezone expansion; explicit semester bounds, local-week quotas and derived series summaries. No duplicated resource eligibility rules.
Existing C++ services / gateway	Reuse room ranking across dates and equipment matching per period. New series operations stay compatible with room allocation JSON v1 and all five strategies.
Existing room / equipment services	Whole or selected-date approvals, scoped ownership checks, exact reservations, occurrence-specific cancellation and physical-loan lifecycle.
Existing waiting-list coordinator	Eligible pre-approved queues retain priority/FCFS. An audited configured higher priority may overtake lower-priority waiters, never confirmed bookings.
Existing events / presentation	Private actor/time history, notifications, calendar projections and reusable forms; no second editable calendar or notification service.

Three necessary shared records

Table	Purpose
reservation_series	One existing room/borrowing parent; immutable recurrence, timezone, semester FK, approval mode and retry identity.
reservation_occurrence	Stable actual interval; Proposed/Unavailable/Rejected/Reserved/Completed/Cancelled outcome, explanation and decision actor/time.
reservation_priority_override	Immutable shared audit: affected request, responsible actor, timestamp, original/selected level, mandatory reason and transaction identity.

Schema: 53 application tables plus Alembic. Migrations 0049a/0049b preserve existing IDs and rows. Existing periods are not guessed to be semesters; pre-migration series can be explicitly bound without redating. FK, range exclusions, aggregate checks and audit guards protect integrity.

Integrity, Verification and Delivery

Important engineering decisions

Partial is consent, not silent degradation. All-required confirmation remains atomic. Partial mode reserves only selected feasible dates and full equipment quantities. Parent summaries are derived; an approved parent never makes every date confirmed. Material changes require a fresh proposal.

Fairness and accountability. Standard priority is zero; additional levels are explicitly configured. Responsible staff overrides require immutable actor/time/original/selected/reason records. Equal eligible queues use FCFS. Pending requests never hold stock or rooms, and priority cannot displace existing reservations or override safety.

Explicit academic policy. Every pickup/start and return/end fits one designated semester. Optional per-student/service weekly minutes and active-series caps have no invented quota defaults. Completed use counts in its calendar week; outstanding overdue possession remains active. Creation and extension increases use policy/student/faculty locking; final approvals recheck current limits.

Preserved history and failure safety. Future cancellation releases exact holds; past outcomes and physical returns remain. Unique outstanding-asset and exclusion constraints prevent double possession/booking. Stale availability, engine errors and failed transactions cannot publish incomplete reservations. Existing patterns are reused pragmatically; no framework or duplicate booking engine was introduced.

Executed verification

Check	Actual result
Fresh Windows C++17 suite	185 passed, 0 failed; two isolated legacy CLI smoke scenarios passed. Existing five allocation strategies remain green.
Full Windows Python / PostgreSQL	482 tests: 481 passed, 1 skipped, 0 failed. Real migration/drift, backup/restore, rollback, concurrency, JSON/CSV parity, existing services and new policies covered.
Final targeted corrections	Safe configured-semester downgrade guard: 1 passed. Consent/semester retry, unchanged retry and concurrent identical retry: 3 passed. These ran after the full Windows suite started.
New policy regression coverage	25 policy cases plus 2 pure date/quota tests on top of earlier recurrence coverage. Per-date decisions, quota races, audit guards, privacy, physical returns and extensions tested.
Linux GitHub Actions	Final source revision: 482 Python/PostgreSQL passed with no skips; 185 freshly compiled C++ tests passed. POSIX permissions and real backup/restore ran. See verified run below.
Real Chrome browser acceptance	1440px desktop and 390px mobile: partial room submission/selected approval/rejection; partial equipment approval with later pending date; private403; no JavaScript errors or page-level overflow.
Schema and artifacts	Mermaid sources/catalog and rendered physical SVG validated against every current table/column. One-page use cases and report rendered and visually checked. No operational data or secrets included.

The Windows POSIX-permission test is skipped because it applies only to Linux. Matching PostgreSQL tools enable real local backup/restore verification. Earlier harness/legacy-fixture failures were corrected and superseded by verified reruns; no assertions were suppressed.

Remaining limits and delivery

Recurring series are not silently split into single-period waiting queues; separate series enrollment still needs an approved policy. Technical bounds remain 366 occurrences and a two-year start-date range inside one configured semester. Execution is synchronous with bounded busy/retry errors. Legacy weekly schedules require explicit calendar bindings. No merge or deployment is included.

Branch: iteration/49-recurring-reservations. Draft PR #13 targets main. Policy implementation: a0e051b; requirements: e7a9215; safe downgrade: cfab74c; consent-aware retries: ec032d9. Earlier recurring-engine and workflow commits are retained in the PR.

Verified CI: https://github.com/najib-attar/resource-allocation-system/actions/runs/39834646243

References: recurrence design and database ER/catalog under docs/architecture/; docs/database-operations.md; docs/engine-protocol.md; updated UC-STU-02/03 requirements and one-page use cases.